System Design Implementation Challenge

Welcome! This challenge is for the practical coding section of our system design interview process.

Goal

We follow the concepts presented in **"System Design Interview – An Insider's Guide" (Vol 1) by Alex Xu** (https://www.amazon.com/System-Design-Interview-insiders-Second/dp/B08CMF2CQF/). While this book is excellent for high-level architecture (drawing boxes and arrows), this challenge focuses on the **implementation**.

Your task is to pick **one** problem from the book, create a new repository, and implement a working prototype.

Rules

- You **MUST** deliver a working, correct, well designed and tested solution.
- You **MAY** pick from the following programming languages: Java, C#, Kotlin, Go, TypeScript, Python. (If you want to do this in a different language, ask your interviewer first)
- You **ARE ENCOURAGED TO** use any AI tool you like (Copilot, ClaudeCode, etc).
- If you don't understand a piece of code the AI builds, make sure to document it clearly with a comment. Any undocumented function, class, or

module will be assumed to be fully understood by you, and you **MUST** be able to answer questions about it. Failure to do so **WILL** result in a lower score.
- For obvious reasons, the pieces of unknown code should be kept to a minimum, if any.

Focus on

- Correct code that builds and passes tests (MANDATORY, non-compiling/test-failing code won't be reviewed)
- Comprehensive tests for the core logic.
- Elegant design (we recommend the APOSD book (https://www.amazon.com/Philosophy-Software-Design-2nd/dp/173210221X/))
- Correct error handling.
- Use of concurrency if needed (don't prematurely optimize).
- Avoiding overengineering.
- Avoiding AI slop.

Nice to haves

- Logging, metrics, ease of deployment.
- Defensive design (timeouts, right-sized connection pools, circuit breakers)
- Good comments / documentation (it's better to write a few yourself than let AI slop over every function)

How to submit 🚀

1. Create a new repository for your solution.
2. Create a folder named after your chosen problem (e.g., `/rate-limiter`).

3. Include a `DESIGN.md` inside that folder explaining your architectural choices, trade-offs, and how you used AI.
4. Send us the link to your new repository.
5. There is no time limit (unless stated so by a member of the team)

Good luck and have fun!

Version: 89f729a